113352019 黃得晉 金融所碩一 機器學習 HW3

Homework3

\usepackage{enumitem}

Split your sample of asset returns into approximately two parts of different time regions: S_{train} and S_{test}. For example, the original sample is N monthly stock returns from January 2015 to December 2024. S_{train} would contain the N stock returns from January 2015 to December 2019, and S_{test} comprises the sample from January 2020 to December 2024.

1. Use S_{train} to extract the factors in the APT model,

$$\tilde{R}_{i,t} = \beta_i^\top f_t + \epsilon_{i,t},$$

where $\tilde{R}_{i,t} = R_{i,t} - \bar{R}_i$ is the de-meaned asset/stock returns and f_t is the K latent factors. To compute f_t, multiply the matrix of returns $R_{i,t}$ (not $R_{i,t}$) with the estimated eigenvectors. Set $K = 8$, compute:

(a) The mean returns of f_t : μ_f

(b) The covariance matrix of f_t : Σ_f

(c) The optimal portfolio returns

$$R_{p,t} = f_t^\top \left(\frac{\Sigma_f^{-1} \mu_f}{\mathbf{1}_N^\top \Sigma_f^{-1} \mu_f} \right),$$

where $\mathbf{1}_N$ is the $N \times 1$ vector of ones.

(d) The mean-to-volatility ratio of $R_{p,t}$. That is, compute the "Sharpe ratio" but without subtracting the risk-free rate. If you also have the risk-free rate, then you can compute the Sharpe ratio instead.

2. Using the estimated eigenvectors from question 1, but now re-compute 1(a) to 1(d) with the sample $\mathcal{S}_{\text{test}}$.

3. Repeat 1 and 2 with $K = 1, ..., 7$. Do you observe increasing mean-to-volatility ratio of $R_{p,t}$ as we increase the number of latent factors in both training and test sample?

```python
import numpy as np
import pandas as pd

# 訓練的資料是所有資料的前80% 測試集是所有資料的後20%
Y_train = df_filled[['年月','證券代碼','報酬率%_月']]
Y_train = Y_train.pivot(index='年月', columns='證券代碼', values='報酬率%_月')
# print(Y_train)
# Y_train = np.nan_to_num((Y_train).values,nan=0.0)
# print(Y_train)
# 前 80% 為訓練集，後 20% 為測試集
n = Y_train.shape[0]
split_idx = int(n * 0.8)
Train_data = np.nan_to_num(Y_train.iloc[:split_idx].copy(),nan = 0.0)
Test_data = np.nan_to_num(Y_train.iloc[split_idx:].copy(),nan = 0.0)

T, N = Train_data.shape
Sigma = np.cov(Train_data, rowvar=False)
# print(Sigma)

# 求解 eigen-decomposition
eigenvalues, eigenvectors = np.linalg.eigh(Sigma)
# print(eigenvalues)
# print(eigenvectors)
idx = np.argsort(eigenvalues)[::-1]  # 降序
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
# print(eigenvalues)
# print(eigenvectors)
K = 8
V_K = eigenvectors[:, :K]  # 取前 K 個 eigenvectors

F_train = Train_data.dot(V_K)
mu_f = np.mean(F_train, axis=0)
Sigma_f = np.cov(F_train, rowvar=False)
# print(mu_f)
# print(Sigma_f)
```

```python
    w_star = np.linalg.inv(Sigma_f).dot(mu_f)
    w_star = w_star / np.sum(w_star)  # normalization
    Rp_train = F_train.dot(w_star)

    sharpe_train = np.mean(Rp_train) / np.std(Rp_train, ddof=1)
    print("Training set (K=8):")
    print("mu_f:\n", mu_f)
    print("Sigma_f:", Sigma_f)
    print("Optimal factor weights (w*):", w_star)
    print("Sharpe ratio :", sharpe_train)
```

✓ 0.0s

```
Training set (K=8):
mu_f:
 [10.9670635  -2.22337202 -0.60987193 -1.31110396 -0.83814024 -1.00547121
 -0.86352935 -1.98934302]
Sigma_f: [[ 3.36687044e+03  6.59737089e-14  3.76992622e-14 -1.69646680e-13
   2.16770758e-13  3.01594098e-13 -1.13097787e-13  3.29868545e-13]
 [ 6.59737089e-14  5.49779019e+02 -4.71240778e-15  1.72002884e-13
  -1.26351434e-13 -1.17810195e-14  3.76992622e-14 -6.59737089e-14]
 [ 3.76992622e-14 -4.71240778e-15  3.95962969e+02 -6.59737089e-14
   1.27235010e-13 -4.00554661e-14 -3.29868545e-14 -3.53062427e-14]
 [-1.69646680e-13  1.72002884e-13 -6.59737089e-14  3.59584835e+02
   8.12890342e-14 -6.59737089e-14 -4.00554661e-14 -1.64934272e-14]
 [ 2.16770758e-13 -1.26351434e-13  1.27235010e-13  8.12890342e-14
   2.96557031e+02 -1.88496311e-14 -1.64566115e-14 -1.76715292e-14]
 [ 3.01594098e-13 -1.17810195e-14 -4.00554661e-14 -6.59737089e-14
  -1.88496311e-14  2.54024623e+02 -7.30423206e-14  1.55509457e-13]
 [-1.13097787e-13  3.76992622e-14 -3.29868545e-14 -4.00554661e-14
  -1.64566115e-14 -7.30423206e-14  2.46372667e+02  4.24116700e-14]
 [ 3.29868545e-13 -6.59737089e-14 -3.53062427e-14 -1.64934272e-14
  -1.76715292e-14  1.55509457e-13  4.24116700e-14  2.32381133e+02]]
Optimal factor weights (w*): [-0.13122175  0.16291678  0.06204773  0.14688506  0.11385453  0.1594541
  0.14119731  0.34486623]
Sharpe ratio : -0.2772009051300006
```

Q2

```
    # Test_data = (Test_data).values
    F_test = Test_data.dot(V_K)
    mu_f_test = np.mean(F_test, axis=0)
    Sigma_f_test = np.cov(F_test, rowvar=False)
    w_star_test = np.linalg.inv(Sigma_f_test).dot(mu_f_test)
    w_star_test = w_star_test / np.sum(w_star_test)
    Rp_test = F_test.dot(w_star_test)
    sharpe_test = np.mean(Rp_test) / np.std(Rp_test, ddof=1)

    print("Testing set (K=8):")
    print("mu_f_test:\n", mu_f_test)
    print("Sigma_f_test:", Sigma_f_test)
    print("Optimal factor weights for testindg data (w*):", w_star_test)
    print("Sharpe ratio :", sharpe_test)
```

✓ 0.0s

```
Testing set (K=8):
mu_f_test:
 [12.75268794 -0.07044579  2.56818237  2.44197363  2.35895887 -2.43553092
  2.7088112   1.31982067]
Sigma_f_test: [[ 1.98554932e+03 -1.70846512e+02  8.15622104e+02  2.90308972e+02
   1.99298867e+02 -2.28384893e+02  2.01147546e+02  2.22472030e+01]
 [-1.70846512e+02  3.30506146e+02  1.64976910e+02 -3.55071498e+01
  -2.20719714e+00  1.87193403e+01  2.98049457e+01  2.62101286e+01]
 [ 8.15622104e+02  1.64976910e+02  9.02496024e+02  1.45025864e+02
   2.03100378e+02 -8.58964841e+01  2.82524404e+02  8.75400872e+01]
 [ 2.90308972e+02 -3.55071498e+01  1.45025864e+02  1.32014989e+02
   4.02335508e+01 -1.43517692e+00  6.38356474e+01  2.15031997e+01]
 [ 1.99298867e+02 -2.20719714e+00  2.03100378e+02  4.02335508e+01
   1.67659524e+02 -2.78063023e+01  7.28981905e+01  6.33096092e+01]
 [-2.28384893e+02  1.87193403e+01 -8.58964841e+01 -1.43517692e+00
  -2.78063023e+01  1.52186599e+02 -2.34552311e+01  1.67252152e+01]
 [ 2.01147546e+02  2.98049457e+01  2.82524404e+02  6.38356474e+01
   7.28981905e+01 -2.34552311e+01  1.98189825e+02  5.85876107e+01]
 [ 2.22472030e+01  2.62101286e+01  8.75400872e+01  2.15031997e+01
   6.33096092e+01  1.67252152e+01  5.85876107e+01  9.09359080e+01]]
Optimal factor weights for testindg data (w*): [ 0.26259529  0.36277288 -0.51977668  0.14040894  0.30854067 -0.17412857
  0.51187772  0.10770975]
Sharpe ratio : 0.475355340558463
```

Q3

```
sharpe_train_list = []
sharpe_test_list = []
K_list = range(1, 9)

for k in K_list:
    if k == 1:
        V_k = eigenvectors[:, :k]
        F_train_k = Train_data.dot(V_k)
        # print(F_train_k)
        mu_f_k = np.mean(F_train_k, axis=0)
        Sigma_f_k = np.cov(F_train_k, rowvar=False)
        Rp_train_k = F_train_k
        sharpe_train_k = np.mean(Rp_train_k) / np.std(Rp_train_k, ddof=1)
        sharpe_train_list.append(sharpe_train_k)
        # print(sharpe_train_k)

        F_test_k = Test_data.dot(V_k)
        mu_f_test_k = np.mean(F_test_k, axis=0)
        Sigma_f_test_k = np.cov(F_test_k, rowvar=False)
        Rp_test_k = F_test_k
        sharpe_test_k = np.mean(Rp_test_k) / np.std(Rp_test_k, ddof=1)
        sharpe_test_list.append(sharpe_test_k)
        # print(sharpe_test_k)
```

```
    else:
        # print(k)
        V_k = eigenvectors[:, :k]
        # print(V_k)
        # print(X_train)
        F_train_k = Train_data.dot(V_k)
        # print("F_train_k:",F_train_k)
        mu_f_k = np.mean(F_train_k, axis=0)
        # print("mu_f_k:",mu_f_k)
        Sigma_f_k = np.cov(F_train_k, rowvar=False)
        # print("Sigma_f_k:",Sigma_f_k)
        w_star_k = np.linalg.inv(Sigma_f_k).dot(mu_f_k)
        w_star_k = w_star_k / np.sum(w_star_k)
        Rp_train_k = F_train_k.dot(w_star_k)
        sharpe_train_k = np.mean(Rp_train_k) / np.std(Rp_train_k, ddof=1)
        sharpe_train_list.append(sharpe_train_k)

        F_test_k = Test_data.dot(V_k)
        mu_f_test_k = np.mean(F_test_k, axis=0)
        Sigma_f_test_k = np.cov(F_test_k, rowvar=False)
        w_star_test_k = np.linalg.inv(Sigma_f_test_k).dot(mu_f_test_k)
        w_star_test_k = w_star_test_k / np.sum(w_star_test_k)
        Rp_test_k = F_test_k.dot(w_star_test_k)
        sharpe_test_k = np.mean(Rp_test_k) / np.std(Rp_test_k, ddof=1)
        sharpe_test_list.append(sharpe_test_k)

print("Results for differ
for k, s_tr, s_te in zip(K_list, sharpe_train_list, sharpe_test_list):
    print(f"K = {k}: Training Sharpe = {s_tr}, Test Sharpe = {s_te}")
```

(variable) K_list: range

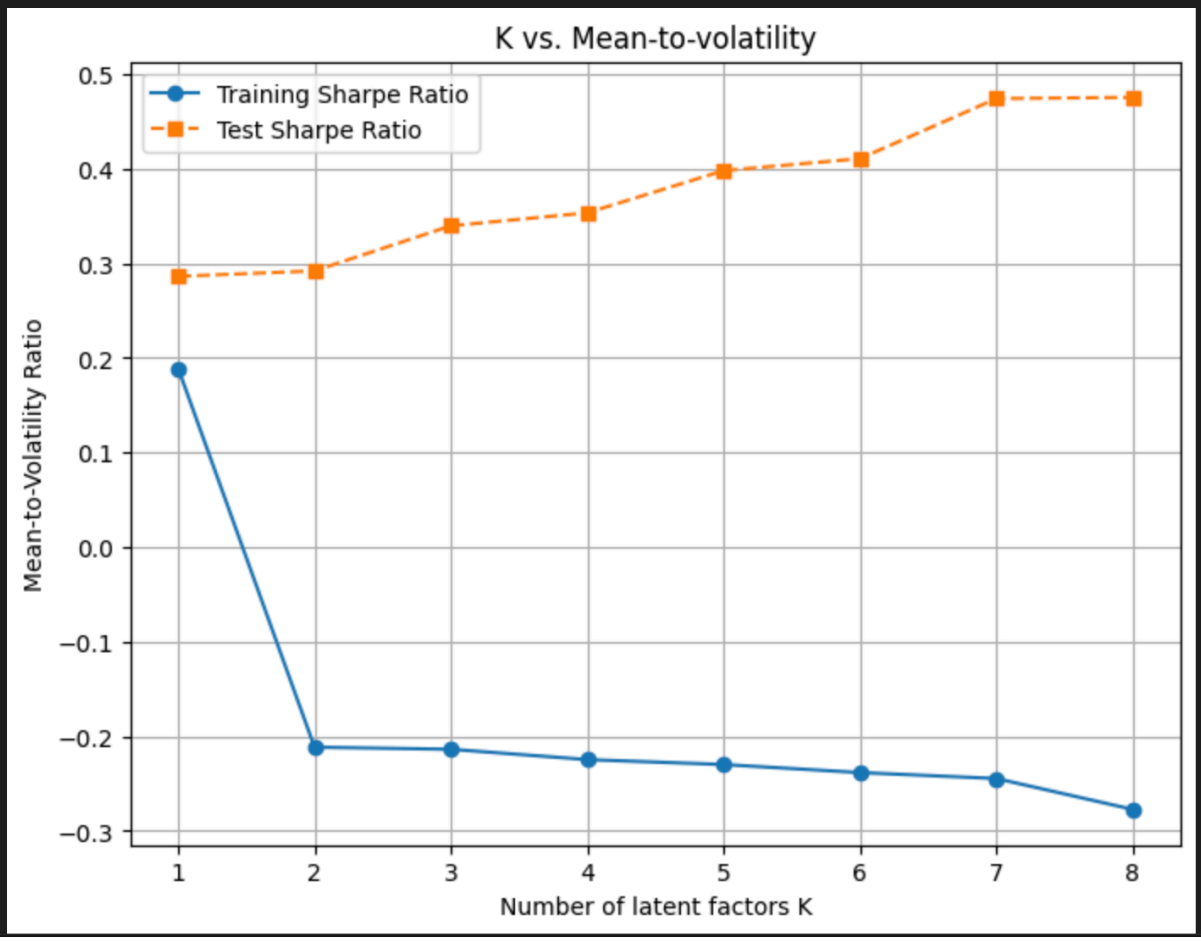
✓ 0.0s

```
Results for different values of K:
K = 1: Training Sharpe = 0.1890066807602238, Test Sharpe = 0.28619457249963204
K = 2: Training Sharpe = -0.21145946984864852, Test Sharpe = 0.29196955066312297
K = 3: Training Sharpe = -0.2136690131733963, Test Sharpe = 0.3395864800061221
K = 4: Training Sharpe = -0.2245772526416016, Test Sharpe = 0.35330427414779797
K = 5: Training Sharpe = -0.22979061062868175, Test Sharpe = 0.39791623731363324
K = 6: Training Sharpe = -0.23829298157048415, Test Sharpe = 0.41060267609020434
K = 7: Training Sharpe = -0.2445612222105519, Test Sharpe = 0.4740477502601916
K = 8: Training Sharpe = -0.2772009051300006, Test Sharpe = 0.475355340558463
```

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plt.plot(K_list, sharpe_train_list, marker='o', linestyle='-', label='Training Sharpe Ratio')
plt.plot(K_list, sharpe_test_list, marker='s', linestyle='--', label='Test Sharpe Ratio')
plt.xlabel('Number of latent factors K')
plt.ylabel('Mean-to-Volatility Ratio')
plt.title('K vs. Mean-to-volatility')
plt.legend()
plt.grid(True)
plt.show()
```

✓ 0.1s



觀察圖表對於 testing data 是有遞增的現象
但是對於 training data 反而是遞減的現象